

# Readora

A quick-commerce bookstore built as 10 Spring Boot microservices, 3 React frontends, and an AI shopping assistant.

This document is a complete walkthrough of the system: what it does well, where the gaps are, the architectural decisions behind it, and a full interview-preparation section covering architecture, data models, and request flows. It is written so that someone who has never seen the codebase can read it once and explain the project confidently.

|                        |                      |                 |                   |
|------------------------|----------------------|-----------------|-------------------|
| 10<br>BACKEND SERVICES | 3<br>REACT FRONTENDS | 8<br>DB SCHEMAS | 9<br>KAFKA TOPICS |
|------------------------|----------------------|-----------------|-------------------|

**Stack** Java 21 · Spring Boot 3.3 · Spring Cloud Gateway · Spring AI · PostgreSQL + pgvector · Redis · Apache Kafka · React 19 · TypeScript · Vite

**Domain** Multi-store quick-commerce · physical + digital goods · payments · returns · delivery logistics · RAG assistant

**Generated** 29 August 2026

# Contents

---

- 01 Executive summary — the 60-second pitch
- 02 System architecture & service catalog
- 03 What the project covers well
- 04 Gaps & what to focus on next
- 05 Design principles, patterns & honest concerns
- 06 Tech stack choices & rationale
- 07 Data model — schemas, entities, relationships
- 08 Key flows — sequence diagrams
- 09 Interview questions & answers
- 10 How to present this project

## 01 • Executive summary

---

The 60-second version you give when someone says "tell me about your project."

**Readora is a quick-commerce bookstore** — think Blinkit or Zepto, but for books. A customer picks a physical store, browses that store's live inventory, and gets a book delivered in ~30 minutes; or they buy a virtual edition and read it instantly in-browser. It is built as **10 independent Spring Boot services** behind a Spring Cloud Gateway, with **three separate React frontends** for the three human roles in the system: customer, store admin, and delivery agent.

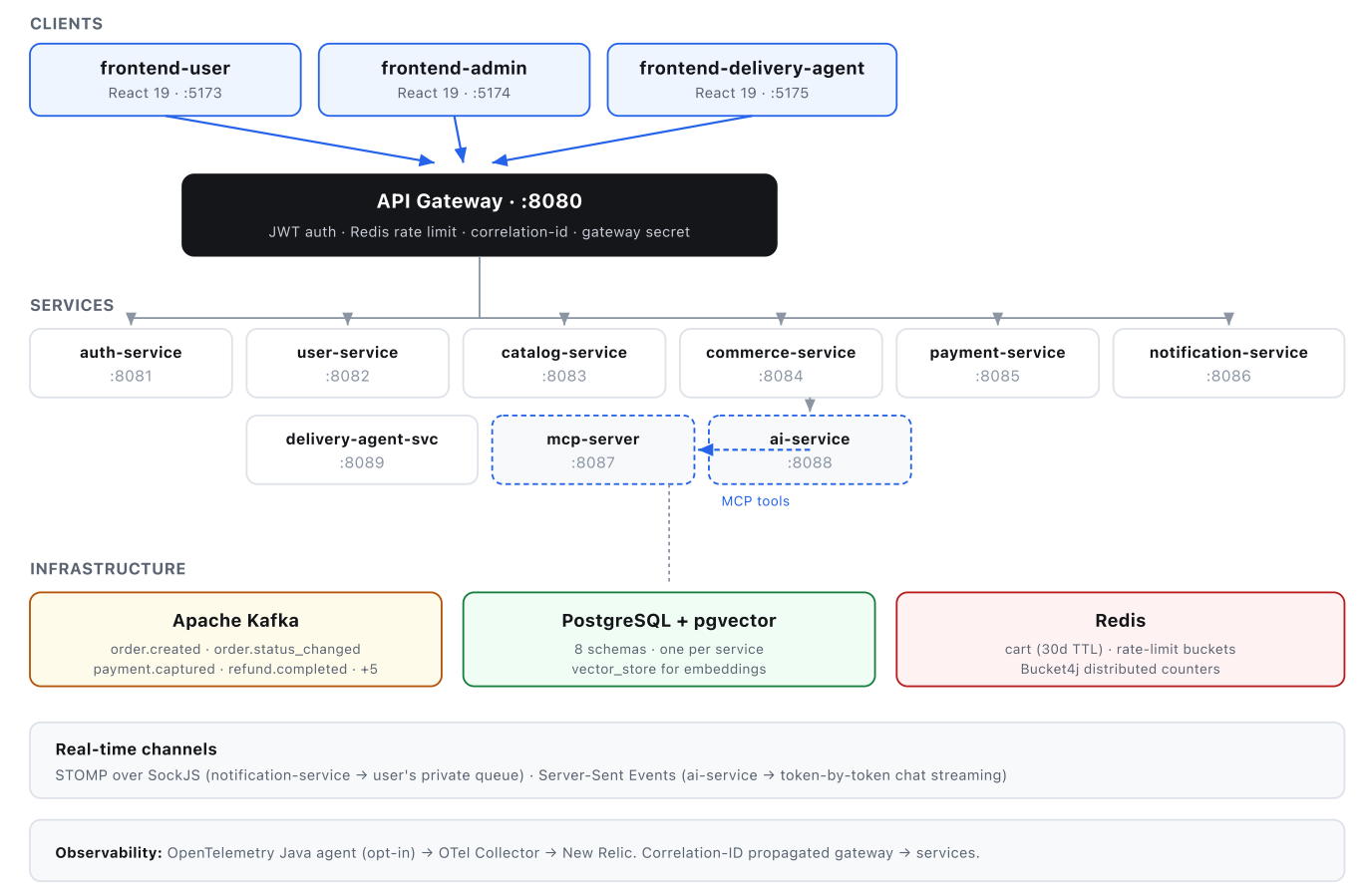
What makes it more than a CRUD app:

- **Genuinely event-driven.** Order → payment → refund → notification all flow over Kafka using the **transactional outbox pattern**, so a database commit and its domain event can never diverge.
- **Two-sided marketplace mechanics.** Delivery agents have their own app with a shared claim queue, optimistic concurrency on claims, and a full status machine that writes back to the order service.
- **An AI assistant with real guardrails.** RAG over pgvector, exposed to the model through Spring AI tools and an MCP server — with server-side enforcement that the model can physically never recommend a book that isn't purchasable at the user's store.
- **Security done properly.** Refresh-token rotation with reuse detection, deny-by-default gateway auth, Redis-backed distributed rate limiting, and a shared-secret filter that blocks direct access to any service that didn't come through the gateway.

### The single best story to lead with

The AI guardrail. It shows judgment, not just wiring: the naive version puts "only recommend available books" in the prompt and hopes. The real version overrides the model's own `storeId` argument server-side before the tool executes, filters candidates through the catalog service, and re-verifies at the end — because an LLM's output is untrusted input. That framing — *treat the model as an untrusted client* — is what separates a demo from an engineering decision.

## 02 · System architecture



Request path (solid) and event path (dashed). Dashed-border services are the AI subsystem.

### Service responsibilities

| SERVICE          | OWNS                                                                                 | NOTABLE                                                                                                         |
|------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| api-gateway      | Routing, edge security                                                               | Deny-by-default JWT validation, Bucket4j+Redis rate limiting, correlation-ID injection, gateway-secret stamping |
| auth-service     | Users, roles, refresh tokens                                                         | Refresh-token <b>rotation with reuse detection</b> ; presenting a revoked token nukes the whole session         |
| user-service     | Profiles, addresses, wallet, wishlist, coupons                                       | Wallet credited/debited by consuming <code>payment.captured</code> and <code>refund.completed</code>            |
| catalog-service  | Books, authors, publishers, categories, inventory, reviews, stores, virtual editions | Store-scoped availability; serves ebook bytes with a path-traversal guard; publishes <code>book.upserted</code> |
| commerce-service | Cart, orders, returns, order status machine                                          | Cart lives in Redis; orders in Postgres; owns the 16-state order lifecycle and the return-review chat           |
| payment-service  | Payments, attempts, refunds                                                          | Simulated UPI settlement via a scheduled job; idempotency guard against Kafka redelivery                        |

| SERVICE                | OWNS                                           | NOTABLE                                                                                  |
|------------------------|------------------------------------------------|------------------------------------------------------------------------------------------|
| notification-service   | Notifications                                  | STOMP/SockJS WebSocket; JWT validated on CONNECT; pushes to /user/queue/notifications    |
| delivery-agent-service | Agents, delivery assignments, return pickups   | Shared claim queue with race handling; calls back into commerce to advance order status  |
| mcp-server             | Model Context Protocol tool surface            | Exposes 6 read-only tools (cart, orders, profile, wallet, book search/detail) to the LLM |
| ai-service             | Chat, conversations, embeddings, vector search | RAG over pgvector; SSE streaming; store-availability guardrail; async embedding backfill |

## 03 · What the project covers well

---

The parts that will genuinely differentiate this from the average portfolio project.

### Transactional outbox pattern

Every domain event is written to an `outbox_events` row inside the same transaction as the business change. A `@Scheduled` relay publishes to Kafka and marks it sent.

- Solves dual-write: the DB commit and the event can't diverge
- Implemented in commerce, payment, and catalog

### Refresh-token rotation + reuse detection

Tokens are hashed at rest. Refreshing revokes the old token and issues a new pair. Presenting an already-revoked token revokes every active token for that user.

- 15-min access token, 30-day refresh
- Textbook OWASP-recommended behavior

### AI guardrails that actually hold

The model's own `storeId` tool argument is overwritten server-side by `StoreScopedToolCallback` before execution — same mechanism as `UserScopedToolCallback` for `userId`.

- Two independent filter layers + fail-closed circuit breaker
- Prompt injection can't escalate scope

### Defense in depth at the edge

Gateway validates JWT and stamps a shared secret. Each service independently re-validates the JWT *and* rejects any request lacking the secret.

- Bypassing the gateway doesn't bypass auth
- Role gating by path prefix per service

### Optimistic concurrency on claims

Two delivery agents racing for the same order: the loser gets a 409 `AssignmentAlreadyClaimed` and the UI reloads the queue.

- Real distributed-systems concern, handled explicitly

### Polyglot persistence, used correctly

Postgres for durable relational data, Redis for ephemeral cart + rate-limit counters, pgvector for embeddings, Kafka for the event log.

- Each store chosen for its access pattern, not fashion

### Resilience4j on cross-service calls

Circuit breakers with deliberate fail-open vs fail-closed semantics per call site — enrichment degrades gracefully, security checks fail closed.

- The distinction is documented in code comments

### Three real frontends, three real roles

Not one app with role toggles — genuinely separate builds with separate auth surfaces, matching how this is deployed in industry.

- Customer, store admin, delivery agent

## Domain modelling depth

The order lifecycle is a **16-state machine** with transitions enforced in the service layer and every hop recorded in `order_status_history`. The return flow alone — request → admin review with a two-party chat → approve/reject → agent pickup assignment → en route → collected → refund initiated → refund completed — is more state modelling than most portfolio projects contain in total.

Ebook entitlement is derived, not stored: access is computed from live order status, so revoking a refunded customer's access requires no separate revocation step and can never drift out of sync.

## 04 · Gaps & what to focus on next

Ranked by how much they'd hurt you in a FAANG-style review. Be honest about these — interviewers respect a candidate who already knows their own weak spots.

### **CRITICAL** There are zero automated tests

Not one unit test, integration test, or E2E spec exists in the repository — across 10 services and 3 frontends. For a project explicitly aimed at FAANG shortlisting, **this is the single biggest liability**. Testing discipline is table stakes at that level, and its total absence is the first thing a senior reviewer will notice.

**What to do, in priority order:**

- **Unit-test the state machines.** `OrderService.updateDeliveryStatus` and `updateReturnPickupStatus` are pure transition logic — trivial to test, high signal. Every illegal transition should throw.
- **Integration-test the outbox → Kafka path** with Testcontainers (Postgres + Kafka). Prove that a rolled-back transaction publishes nothing.
- **Test the AI guardrail.** Assert that a model-supplied `storeId` is discarded and the caller's real one is used. This is your headline feature — prove it works.
- **One Playwright E2E:** browse → add to cart → checkout → order appears. `.gitignore` already anticipates Playwright, so the intent was there.

### **CRITICAL** No database migration tooling

Every service runs `spring.jpa.hibernate.ddl-auto: update`. There is no Flyway or Liquibase anywhere. This works locally but is indefensible in a production conversation: no rollback story, no reviewable schema diff, no way to reproduce a schema deterministically, and Hibernate silently refuses some changes (column narrowing, type changes) rather than failing loudly.

**Fix:** introduce Flyway, baseline the current schema from `dataset/seed.sql` as `V1__baseline.sql`, and switch to `ddl-auto: validate`. This is a weekend of work and removes an entire class of interview objection.

### **HIGH** No CI pipeline

No GitHub Actions workflow, no build badge, no automated check on push. Even a minimal pipeline that runs `mvn verify` across the reactor and `tsc --noEmit` on each frontend would demonstrate release discipline — and would have caught the two issues below automatically.

### **HIGH** The outbox relay is not safe to scale horizontally

`OutboxRelay` polls `findAllByPublishedAtIsNull()` on a fixed 2-second delay with no row locking. Run two instances of commerce-service and both will read the same unpublished rows and publish the same event twice. Consumers are mostly idempotent, so it degrades rather than breaks — but the pattern is incomplete as written.

**Fix:** `SELECT ... FOR UPDATE SKIP LOCKED` on the claim query, or `ShedLock` for single-instance scheduling. Mentioning this unprompted in an interview is a strong signal.

### HIGH Containerisation is incomplete

Only Redis, Kafka, and the OTel collector are containerised. Postgres runs natively and no service has a `Dockerfile`. Running the system means ten manual `mvn spring-boot:run` invocations in ten terminals. A reviewer who wants to try it will likely give up.

**Fix:** a multi-stage `Dockerfile` per service and one `docker compose up` that brings up the whole system. This has the highest ratio of "reviewer impression" to "engineering effort" of anything on this list.

### MEDIUM Correlation IDs stop at the Kafka boundary

`CorrelationIdFilter` propagates a trace ID across HTTP hops, but events published to Kafka don't carry it in their headers. A trace therefore breaks exactly where the interesting asynchronous work begins — the checkout → payment → notification chain is invisible end-to-end.

### MEDIUM Schema-per-service, not database-per-service

All ten services point at one Postgres instance ( `jdbc:postgresql://localhost:5432/readora` ) and are separated by schema. This is a legitimate pragmatic choice — and worth defending as such — but it is *not* full service isolation: one runaway query can starve every service, and nothing technically prevents a cross-schema join. Know the difference and name it before the interviewer does.

### MEDIUM Smaller items worth a sweep

- **N+1 in admin agent tracking.** `AdminDeliveryService.listAgents()` issues two queries per agent to find active work. Fine at 3 agents, wrong at 300 — replace with a single grouped query.
- **Unbounded list endpoints.** Authors, publishers, and categories return everything with no pagination; the frontend filters client-side.
- **Payment provider is simulated.** Honest and clearly documented in code, but be ready to explain what a real PSP integration adds: webhooks, signature verification, 3DS, reconciliation.
- **Cart has no durable fallback.** Redis is the sole system of record; a flush loses every active cart.
- **No architecture docs in-repo.** The README covers running the system but not how it works — this document should be adapted into `docs/`.

### Not a gap — credit where due

Secrets are handled correctly. `.gitignore` excludes every `.env`, and only `.env.example` templates are tracked. No credentials are committed. That is a genuinely common failure in portfolio repos and this one gets it right.

# 05 · Design principles, patterns & concerns

## Patterns deliberately applied

| PATTERN                           | WHERE                                                                                     | WHY IT WAS THE RIGHT CALL                                                                             |
|-----------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Transactional Outbox              | <code>OutboxEvent</code> + <code>OutboxRelay</code> in commerce, payment, catalog         | Removes the dual-write problem between DB and broker without needing distributed transactions         |
| Saga (choreography)               | Checkout → payment → order confirmation → notification                                    | No central orchestrator; each service reacts to events. Compensation = refund on cancel/return        |
| API Gateway                       | Spring Cloud Gateway                                                                      | One place for auth, rate limiting, CORS, tracing — services stay focused on domain logic              |
| Decorator                         | <code>UserScopedToolCallback</code> , <code>StoreScopedToolCallback</code>                | Wraps an LLM tool call to forcibly rewrite an argument — security enforced without trusting the model |
| Circuit Breaker                   | Resilience4j in commerce + ai-service                                                     | Prevents cascading failure; fail-open for enrichment, fail-closed for correctness                     |
| Snapshot / denormalisation        | <code>OrderItem</code> freezes title, ISBN, price at purchase                             | An order must stay historically accurate even if the catalog changes later                            |
| State machine                     | <code>OrderStatus</code> + guarded transitions                                            | Illegal transitions throw rather than silently corrupting order state                                 |
| Repository / Service / Controller | Uniform across all 10 services                                                            | Predictable layering; a reviewer can navigate any service after reading one                           |
| Idempotency key                   | Checkout accepts <code>Idempotency-Key</code> ; payments keyed on <code>order:{id}</code> | A retried checkout returns the original order instead of double-charging                              |
| Anti-corruption layer             | Mirrored DTOs per consuming service                                                       | Each service owns its own copy of a contract instead of sharing a coupling-heavy common module        |

## Principles held consistently

- **Single source of truth per fact.** `Order.status` in commerce-service is authoritative; delivery-agent-service keeps its own *bookkeeping* copy for the agent UX and is explicitly documented as non-authoritative.
- **Derived over stored, where derivation is cheap.** Ebook entitlement is computed from order status rather than maintained as a separate table that could drift.
- **Fail closed on anything security-shaped.** Admin store resolution, availability checks, and the final book-ID filter all return "nothing" on error rather than "everything."
- **Comments explain *why*, not *what*.** The codebase is unusually strong here — non-obvious decisions carry the reasoning inline. This will read very well to a reviewer.

## Architectural concerns worth pre-empting



**Shared JWT secret across all services**

Every service validates tokens with the same symmetric HMAC secret. Compromise one service and you can mint tokens for all of them. Production answer: asymmetric signing (RS256) where auth-service holds the private key and others verify with a public key via JWKS.

**Synchronous coupling on the read path**

An order-detail request fans out to payment-service and catalog-service. If either is slow, the page is slow. Circuit breakers cap the damage, but the deeper fix is a read model / CQRS projection that keeps a denormalised view fed by the events already flowing.

**Kafka consumers assume at-least-once without uniform idempotency**

Payment refunds guard explicitly against redelivery; some other consumers rely on state checks that happen to be idempotent. It works, but the guarantee isn't applied uniformly — worth auditing consumer by consumer and stating the strategy explicitly.

## 06 · Tech stack choices & rationale

For each choice: what it is, why it beats the obvious alternative, and the trade-off you accepted.

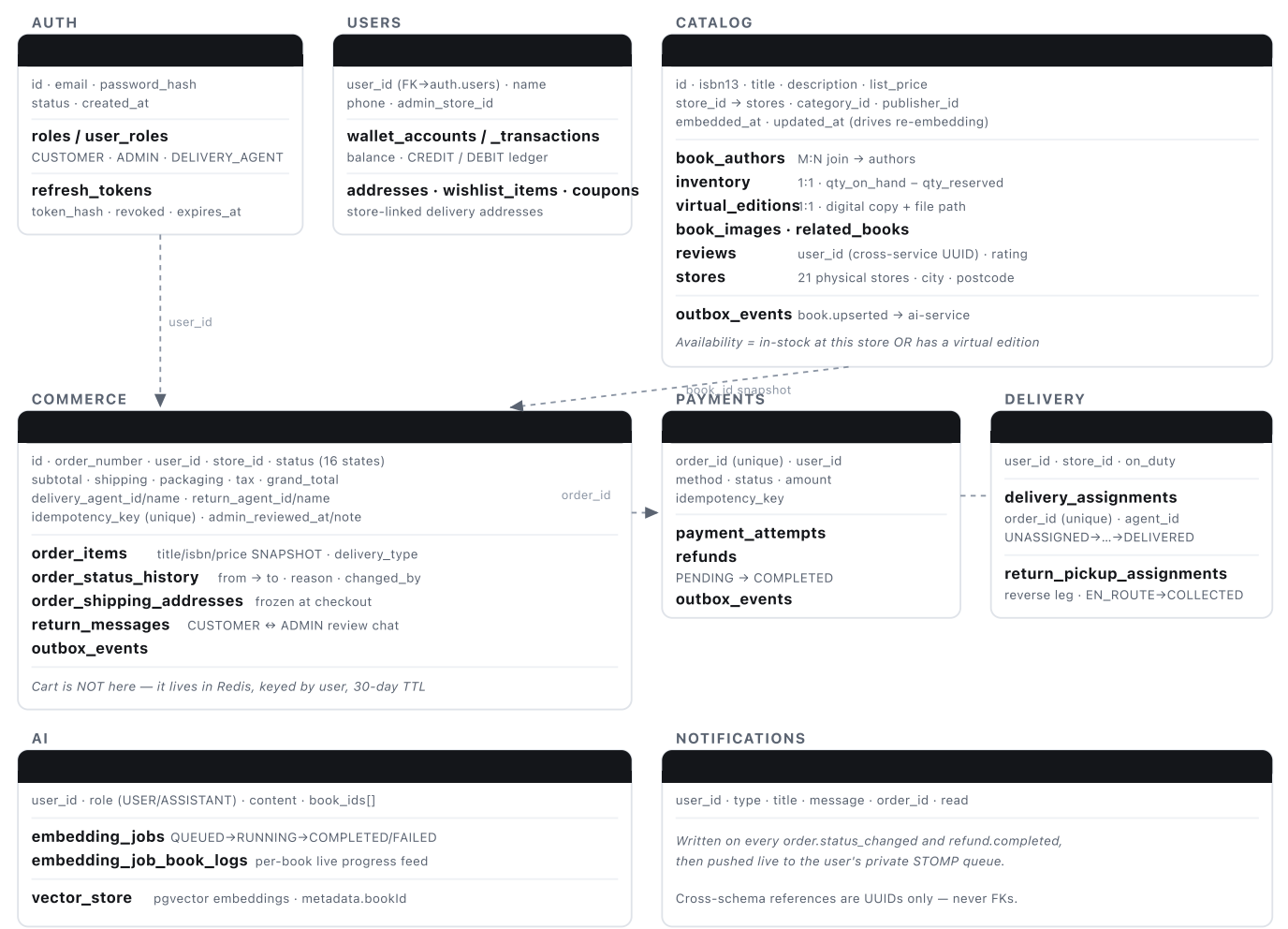
| CHOICE                       | WHY                                                                                                                                                                          | TRADE-OFF YOU ACCEPTED                                                                                                                             |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Java 21 + Spring Boot 3.3    | Records, pattern-matched switches, and virtual-thread readiness. Boot 3.x is the current LTS-aligned line with native Jakarta EE 10.                                         | Heavier runtime than Go/Node; slower cold start. Irrelevant for long-running services.                                                             |
| Microservices (10)           | The point of the project is demonstrating distributed-systems skill: independent deploy units, event choreography, per-service data ownership.                               | Genuine overkill for the traffic. <i>Say this out loud in interviews</i> — knowing when a modular monolith is the right answer is a senior signal. |
| Spring Cloud Gateway         | Reactive, non-blocking, first-class Spring integration for global filters (JWT, rate limit, correlation ID).                                                                 | Ties the edge to the Spring ecosystem versus a language-agnostic Envoy/Kong.                                                                       |
| Apache Kafka                 | Durable, replayable, multi-consumer log. Multiple services independently consume <code>order.status_changed</code> without knowing about each other.                         | Heavy operationally. RabbitMQ would be simpler if replay and fan-out weren't needed.                                                               |
| PostgreSQL + pgvector        | One engine for relational data <i>and</i> vector similarity search — no separate Pinecone/Weaviate to operate, and embeddings sit next to the data they describe.            | Won't match a dedicated vector DB at very large scale. At 53 books it isn't close to mattering.                                                    |
| Redis                        | Two distinct jobs: cart (ephemeral, TTL-friendly, no reporting need) and distributed rate-limit counters via Bucket4j.                                                       | Cart durability depends entirely on Redis persistence config.                                                                                      |
| Spring AI + MCP              | Model-agnostic abstraction over chat/embeddings/tools. MCP is the emerging standard for exposing tools to LLMs — using it shows currency with where the industry is heading. | Spring AI 1.0 is young; APIs still shift between versions.                                                                                         |
| React 19 + TypeScript + Vite | Fast HMR, strict typing across three apps, mature ecosystem.                                                                                                                 | No SSR — irrelevant for authenticated app surfaces, would matter for SEO on public catalog pages.                                                  |
| Redux Toolkit                | Genuinely global cross-cutting state (auth, cart, selected store, notifications) consumed by many unrelated components.                                                      | More ceremony than Zustand/Context; justified by the breadth of shared state here.                                                                 |
| OpenTelemetry                | Vendor-neutral instrumentation via a zero-code Java agent, exported through a collector to New Relic.                                                                        | Opt-in rather than always-on; the collector is one more moving part.                                                                               |

### The question you will definitely be asked

*"Why microservices for a project this size?"* — The strong answer is not a defence. It is: **"Because the goal was to demonstrate distributed-systems competence, and I'd choose a modular monolith for a real product at this scale."** Then show you understand the cost you paid: network hops on the read path, eventual consistency the UI has to absorb, ten deploy units, and distributed debugging. Candidates who defend microservices unconditionally look junior; candidates who can price the trade-off look senior.

## 07 · Data model

Eight schemas in one PostgreSQL instance, one per service. No foreign keys cross a schema boundary — cross-service references are plain UUID columns, never JPA relationships.



Solid boxes are schemas; dashed arrows are cross-service UUID references (no database-level foreign key).

### Relationships worth being able to recite

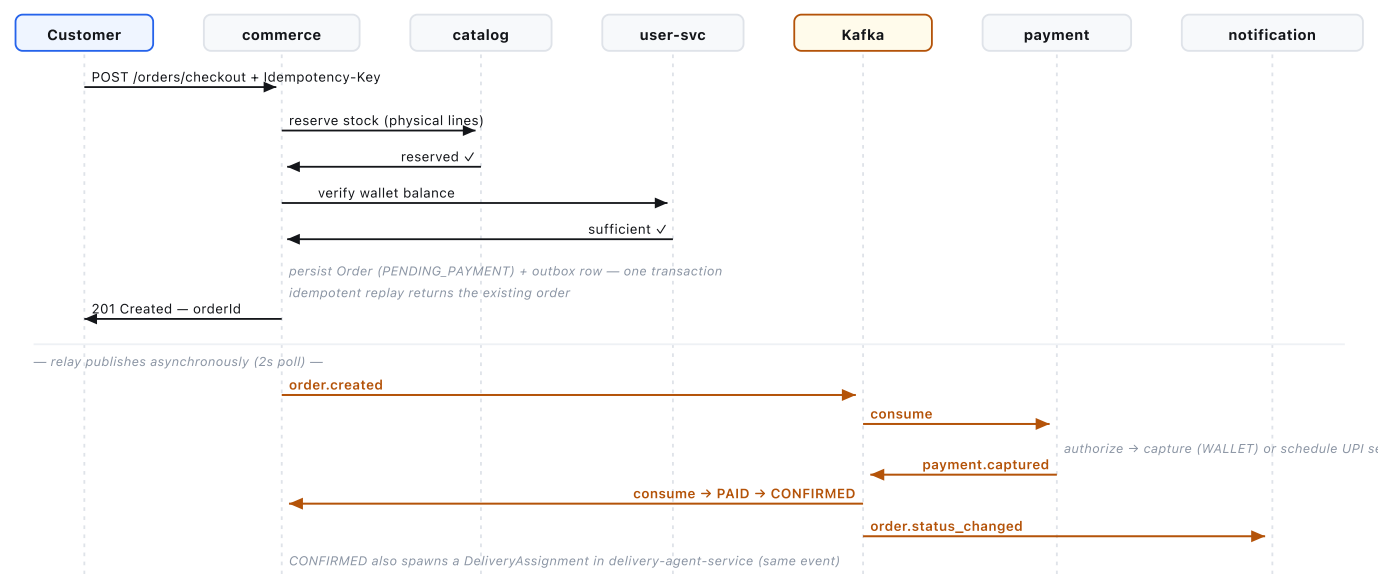
| RELATIONSHIP          | CARDINALITY | WHY IT'S MODELLED THAT WAY                                                                                |
|-----------------------|-------------|-----------------------------------------------------------------------------------------------------------|
| Book ↔ Author         | M : N       | Join table <code>book_authors</code> ; a book can have co-authors, an author many books                   |
| Book → Store          | N : 1       | Each physical copy belongs to exactly one store — this is what makes availability store-scoped            |
| Book → Inventory      | 1 : 1       | Available = <code>qty_on_hand - qty_reserved</code> ; reservation happens at checkout, not at add-to-cart |
| Book → VirtualEdition | 1 : 0..1    | Optional digital twin. Store-independent — a virtual edition is buyable from anywhere                     |
| Order → OrderItem     | 1 : N       | Each line carries a <b>snapshot</b> of title/ISBN/price so history stays accurate                         |
| Order → StatusHistory | 1 : N       | Append-only audit of every transition, with who caused it                                                 |

| RELATIONSHIP                   | CARDINALITY | WHY IT'S MODELLED THAT WAY                                                                         |
|--------------------------------|-------------|----------------------------------------------------------------------------------------------------|
| Order → Payment                | 1 : 1       | Unique constraint on <code>order_id</code> ; enforced across a service boundary by idempotency key |
| Payment → Refund               | 1 : 0..N    | Modelled for partial refunds; currently only full refunds are issued                               |
| Order → DeliveryAssignment     | 1 : 0..1    | Only physical orders get one, created reactively when the order reaches CONFIRMED                  |
| Order → ReturnPickupAssignment | 1 : 0..1    | Separate entity from delivery — the reverse leg can exist after delivery already completed         |
| User → WalletAccount           | 1 : 1       | Balance plus an append-only <code>wallet_transactions</code> ledger                                |
| Conversation → Message         | 1 : N       | Assistant messages carry a <code>book_ids[]</code> array so carousels survive a page reload        |

## 08 · Key flows

Three flows worth being able to draw on a whiteboard from memory.

### 8.1 · Checkout — the saga



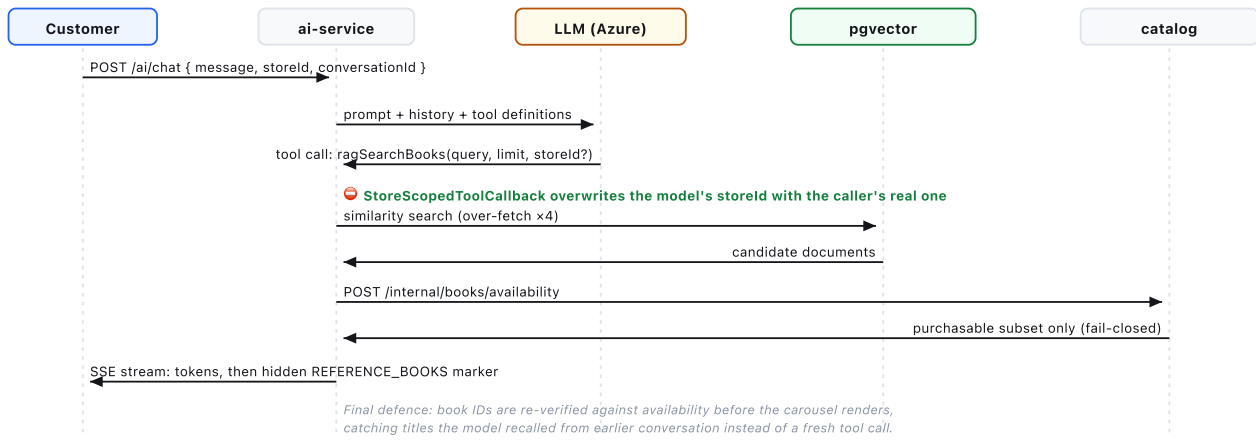
Choreographed saga: no orchestrator. Each service reacts to events and publishes its own.

### 8.2 · Return with human review



Eight-state return workflow. Refund completion is event-confirmed, never assumed.

### 8.3 · AI chat with RAG and the store guardrail



The model is treated as an untrusted client: its arguments are overridden, its outputs re-verified.

## 09 · Interview questions & answers

---

Grouped by what the interviewer is actually probing for. Answers are compressed — expand them in your own words.

### ARCHITECTURE & SYSTEM DESIGN

#### Q Walk me through what happens when a user places an order.

Gateway validates the JWT and stamps a correlation ID and gateway secret. commerce-service reserves stock through catalog-service for physical lines and verifies wallet balance through user-service — both synchronous, because a failure here must block the order. It then writes the Order and an outbox row **in one transaction** and returns 201 immediately. A relay publishes `order.created` ; payment-service consumes it, captures (wallet) or schedules settlement (UPI), and publishes `payment.captured` ; commerce-service moves the order PAID → CONFIRMED; that status event fans out to notification-service and delivery-agent-service, which creates a claimable assignment.

#### Q Why the outbox pattern instead of just publishing to Kafka in the service method?

The dual-write problem. If I save the order and then publish, a crash between the two loses the event — the order exists but nothing downstream knows. If I publish first and the save fails, I've announced an order that doesn't exist. Kafka can't join a database transaction. The outbox makes the event part of the same commit, so they're atomic; a separate relay handles delivery with at-least-once semantics, which consumers absorb through idempotency.

#### Q Choreography or orchestration? Why?

Choreography. Each service reacts to events and publishes its own, with no central coordinator. Good: services stay decoupled, adding a consumer needs no changes upstream. Bad: no single place to see the flow, and debugging is harder. If the saga grew to eight or ten steps with complex compensation, I'd move to orchestration — a Saga Execution Coordinator makes the flow explicit and compensation ordering controllable.

#### Q How do you guarantee exactly-once processing?

I don't — I guarantee at-least-once delivery plus idempotent consumers, which is the practical equivalent and far cheaper. Payment-service checks whether a refund already exists for a payment before creating one. Order status transitions are guarded by the current state, so replaying a stale event is a no-op. Checkout is protected by a client-supplied `Idempotency-Key` with a unique constraint. Exactly-once across a broker and a database requires distributed transactions, which cost more than they're worth here.

#### Q What happens if catalog-service is down during checkout?

The Resilience4j circuit breaker on `CatalogClient` opens and checkout fails fast with a clear 503 rather than hanging. That's deliberate — stock reservation is correctness-critical, so it fails closed. Contrast that with cover-image lookup on the orders list, which fails *open*: a missing thumbnail is cosmetic, so it degrades to a placeholder rather than breaking the page. Choosing per call site is the actual design decision.

#### Q Why is the cart in Redis and not Postgres?

Access pattern. A cart is high-write, low-value, per-user, and needs natural expiry — one JSON blob under a user key with a 30-day TTL. No reporting query ever runs over carts, and losing one is an inconvenience, not a data-integrity failure. Orders are the opposite: durable, queried, audited, so they belong in Postgres. The honest trade-off is that a Redis flush drops every live cart; production would enable AOF persistence or accept the loss explicitly.



### Q Two delivery agents tap "Accept" on the same order simultaneously. What happens?

Both requests load the assignment; both see `UNASSIGNED`. The first to commit claims it. The second hits the status guard and gets a 409 `AssignmentAlreadyClaimed`, and the UI reloads the queue so the order simply disappears. Under real contention I'd add optimistic locking with an `@Version` column so the losing write fails at the database rather than depending on transaction timing.

## DATA MODELLING

### Q Why does OrderItem duplicate the book's title and price?

Deliberate denormalisation for temporal accuracy. An order is a historical record of what someone bought at a moment in time. If the price changes or a book is renamed or deactivated, the order must still show what was actually purchased and charged. Joining live catalog data would silently rewrite history — and an invoice that changes retroactively is a real-world correctness bug, not a style preference.

### Q There are no foreign keys between schemas. Isn't that a data-integrity risk?

Yes, and it's the accepted cost of service autonomy. A foreign key from `commerce.orders` to `auth.users` would couple the two services at the storage layer — you couldn't deploy or migrate them independently, and you certainly couldn't split them onto separate databases later. Cross-service references are plain UUIDs, and consistency is maintained at the application layer. Since the services already share one Postgres instance, this discipline is what keeps the future option of true separation open.

### Q How do you model a book that's both physical and digital?

One `Book` row with an optional 1:1 `VirtualEdition`. The physical copy belongs to exactly one store and has an `Inventory` row; the virtual edition is store-independent and has no stock. Availability is therefore: in stock at *this* store, **or** has an active virtual edition. That single rule is centralised and reused by the product page, checkout, and the AI guardrail — which is exactly why the guardrail was cheap to add.

### Q How does ebook access revocation work after a refund?

It's derived, not stored. When the reader requests content, catalog-service asks commerce-service for the user's purchased book IDs, and that query excludes orders in cancelled or return-approved-onward states. So access is recomputed on every request — there's no entitlement row to revoke and nothing that can drift. The nuance is *where* the cut-off sits: access survives while a return is merely requested and under review, and is only revoked once an admin approves. Rejecting the return restores nothing because nothing was ever taken away.

## THE AI SUBSYSTEM

### Q How does the RAG pipeline work?

Book text — title, authors, description, table of contents — is embedded and stored in pgvector with the book ID in metadata. At query time the user's message drives a similarity search that returns candidate documents, which are injected into the model's context as tool results. The model reasons over real catalog data instead of hallucinating from training memory. Re-embedding is incremental: saving a book publishes `book.upserted` and only that title is re-indexed; a full backfill compares `embedded_at` against `updated_at` so it never redoes current work.

### Q Your assistant recommended a book that wasn't buyable. How did you fix it?

This is my favourite part of the project. The naive fix is a prompt instruction — which is a suggestion, not a guarantee. Instead I treated the model as an untrusted client. Every retrieval tool takes a `storeId`, but a decorator (`StoreScopedToolCallback`) **overwrites whatever the model supplied** with the caller's real store before the tool executes, so an invented or omitted value can't widen scope. The tool then filters candidates through catalog-service's availability endpoint, over-fetching 4x so filtering doesn't starve results. That call is fail-closed: if catalog-service is down we recommend nothing rather than something wrong. Finally, the book IDs the model cites are re-verified before rendering — which catches a title recalled from earlier conversation instead of a fresh tool call.

## Q What is MCP and why use it?

Model Context Protocol is an emerging open standard for exposing tools to LLMs over a defined transport, rather than each application inventing its own function-calling glue. Here, mcp-server exposes six read-only tools — cart, order history, profile, wallet, book search, book detail. The security-relevant part is that user-scoped tools take a `userId` parameter that is *never* trusted from the model: `UserScopedToolCallback` rewrites it with the authenticated caller's ID. Without that, a prompt-injected model could ask for someone else's cart.

## Q Why stream over SSE instead of WebSocket?

Chat completion is unidirectional — server to client, one response at a time. SSE gives that with plain HTTP: no upgrade handshake, automatic reconnection, works through proxies. WebSocket would be over-engineered. Notifications *are* on WebSocket (STOMP over SockJS) because those are server-initiated pushes to a long-lived per-user subscription, which is genuinely bidirectional-shaped. Different problems, different transports.

## SECURITY

### Q Explain your authentication flow.

Login returns a 15-minute access token and a 30-day refresh token; only a hash of the refresh token is stored. Refreshing **rotates**: the presented token is revoked and a new pair issued, so a stolen token is usable at most once. If an already-revoked token is presented — the signature of theft — every active token for that user is revoked, killing the session everywhere. The gateway validates JWTs deny-by-default, and each service re-validates independently rather than trusting an upstream header.

### Q If the gateway already authenticates, why validate again in every service?

Defence in depth. Services are reachable on their own ports; anything that could reach them directly would bypass the edge entirely. So each service re-validates the JWT *and* requires a shared gateway secret, rejecting anything that didn't come through the gateway or another internal service. It also keeps services independently testable and deployable without a gateway in front of them.

### Q How does rate limiting work across multiple gateway instances?

Bucket4j backed by Redis, so the token bucket is shared rather than per-instance — otherwise N instances would multiply the effective limit by N. Limits are per-caller with per-route overrides; ai-service is capped far tighter than the rest because each call costs real money at an inference API. One operational gotcha I documented: buckets bake in their limit at creation, so changing config requires flushing the Redis keys, not just restarting.

### Q Tell me about a security bug you found in your own code.

An IDOR on `GET /api/v1/payments/{orderId}`. The route required authentication, so it looked safe — but it never checked that the caller *owned* the order. Any logged-in user could read another user's payment amount and method by guessing an order ID. Authentication isn't authorisation. The fix scopes the lookup to the caller and returns the same 404 whether the payment doesn't exist or isn't theirs, so you can't probe for valid IDs. I kept a separate unchecked method for internal service-to-service use where ownership is already verified upstream.

## REFLECTION — THE QUESTIONS THAT SEPARATE CANDIDATES

### Q What would you do differently if you started over?

- **Tests from day one.** Retrofitting them into 10 services is painful; the state machines and the guardrail should have been TDD'd.
- **Flyway from commit one.** `ddl-auto: update` was convenient early and is now a liability.
- **Start as a modular monolith** with clean internal boundaries, and extract services only where independent scaling or deployment actually justified it.
- **Trace context in Kafka headers** from the start, so async flows are observable end-to-end.

### Q What's the weakest part of this system?

The complete absence of automated tests — I'd say it before being asked. Every behaviour is verified by hand, which doesn't scale and gives no regression safety. The second weakest is the outbox relay: correct for a single instance, but it polls without row locking, so two instances would double-publish. `SELECT ... FOR UPDATE SKIP LOCKED` fixes it, and I know precisely where the change goes.

### Q How would you scale this to 10,000 orders a minute?

- **Fix the relay first** — it's the correctness blocker for running multiple instances.
- **Partition Kafka by order ID** to keep per-order ordering while parallelising consumers.
- **Split the database** — schema-per-service becomes database-per-service; commerce and catalog are the hot paths.
- **Add a read model.** Order detail currently fans out synchronously to two services; a CQRS projection fed by existing events removes that from the request path.
- **Cache the catalog** — book detail is read-heavy and changes rarely.
- **Move stock reservation to optimistic locking or a queue;** row contention on popular titles becomes the bottleneck before anything else does.

### Q What was the hardest bug you debugged here?

Two worth telling. First: the assistant's book carousel vanished on new conversations. The cause wasn't in the chat code at all — the layout keyed its page transition on `location.pathname`, so navigating from `/assistant` to `/assistant/:id` looked like a new page, unmounted the component, and wiped in-memory state. Second: markdown lists rendered correctly after a reload but not while streaming. SSE splits multi-line payloads across several `data:` lines within one frame, and I was emitting per line instead of rejoining with `\n` per frame — so every newline silently disappeared from live output. Both taught the same lesson: the bug is often one layer away from where it shows up.

# 10 · How to present this project

## The 60-second version

"Readora is a quick-commerce bookstore — books delivered in about 30 minutes, or read instantly as an ebook. It's ten Spring Boot microservices behind a Spring Cloud Gateway with three React frontends for customers, store admins, and delivery agents. The parts I'd point at: order and payment flow through Kafka using the transactional outbox pattern so a database commit and its event can't diverge; there's a full return workflow with human review and a delivery-agent pickup leg; and there's an AI shopping assistant doing RAG over pgvector with server-side guardrails that make it structurally impossible for the model to recommend a book you can't actually buy at your store."

## Before the interview

- **Redraw all three sequence diagrams from memory.** If you can whiteboard checkout without notes, you own the system.
- **Rehearse the guardrail story.** It's your strongest — problem, naive fix, why it's insufficient, real fix, layers of defence.
- **Memorise the four numbers:** 10 services, 3 frontends, 8 schemas, 9 Kafka topics.
- **Have the IDOR story ready.** "I found a security bug in my own code" is a strong, unprompted signal.
- **Practise saying "microservices were overkill here."** Volunteering the trade-off reads as senior; defending it unconditionally reads as junior.

## Highest-leverage work before you show this to anyone

| # | TASK                                               | EFFORT     | WHY IT MOVES THE NEEDLE                                                 |
|---|----------------------------------------------------|------------|-------------------------------------------------------------------------|
| 1 | Tests on the state machines + guardrail            | 2–3 days   | Removes the single biggest objection; proves the headline feature works |
| 2 | <code>docker compose up</code> runs everything     | 1–2 days   | A reviewer who can't run it in one command probably won't run it at all |
| 3 | Flyway migrations, <code>ddl-auto: validate</code> | 1 day      | Turns an indefensible choice into a professional one                    |
| 4 | GitHub Actions running build + typecheck           | Half a day | Visible release discipline; a green badge on the README                 |
| 5 | Architecture docs + diagrams in <code>docs/</code> | 1 day      | Most reviewers read the README and never open the code                  |
| 6 | <code>SKIP LOCKED</code> on the outbox relay       | 2 hours    | Small change, and a great story about knowing your own limitations      |

### Final read

The architecture, domain modelling, and AI engineering here are genuinely above the bar for a portfolio project — the return workflow and the guardrail design in particular are things most candidates simply don't have. The gap is engineering *process*, not engineering *ability*: tests, migrations, CI, and one-command startup. Those are four well-understood items, roughly a week of focused work, and none of them require new design thinking. Close them and this stops being a strong student project and starts reading like production work.